

实 验 报 告

（与程序设计有关的课程用）

课程名称：软件工程

实验题目：实验二　图书管理系统应用程序设计

班级学号：XXXXXXXXXX

姓　　名：XXX

成　　绩：

2026 年 10 月 16 日

一、实验目的及要求

（一）实验目的

1. 掌握用高级程序设计语言访问关系数据库的方法，理解 JDBC 的工作原理与编程步骤；

2. 掌握在应用程序中调用数据库存储过程、视图的方法，理解数据库完整性约束在应用程序中的体现；

3. 掌握图形用户界面程序的常用设计方法，理解表示层、业务逻辑层、数据访问层的分层设计思想；

4. 完成图书管理系统应用程序的设计、编码、调试与测试，形成完整的软件工程实践能力。

（二）实验要求

在实验一建立的图书管理系统数据库 library_db 的基础上，用 Java 语言开发图书管理系统的应用程序，具体要求如下：

1. 通过 JDBC 连接 MySQL 数据库，连接参数写入配置文件，不硬编码在源程序中；

2. 实现图书信息和读者信息的管理功能，包括信息的录入、修改、删除和查询；

3. 实现借书与还书的办理，并能够查看借阅明细与未归还清单；

4. 提供借阅数据的统计查询功能，并以表格和图形两种方式显示统计结果；

5. 程序应能正确处理数据库返回的完整性约束错误，并给出友好的中文提示。

（三）关于开发平台的说明

实验指导书给出的示例平台为 Microsoft Access 或 Visual Basic 6.0。本次实验在 Windows 11 环境下改用 Java 21 + JDBC + Swing 实现，数据库仍使用实验一建立的 MySQL 8.0.44 实例 library_db。这样做的原因是：Java 是当前企业级应用开发的主流语言，JDBC 是访问关系数据库的标准接口，采用 Java 实现既符合课程要求，又能与实验一的数据库设计成果自然衔接。

二、软、硬件环境

（一）硬件环境

项目	配置
处理器	AMD Ryzen 9 7945HX（16 核 32 线程）
内存	16 GB（系统可用 15.7 GB）
硬盘	Samsung MZVL21T0HCLR-00BL2 固态硬盘，954 GB

（二）软件环境

类别	软件名称	版本

操作系统	Microsoft Windows 11 家庭版（64 位）	10.0.26200
开发语言	Oracle JDK（JAVA_HOME = D:\Develop\JDK21）	21.0.9 LTS
构建工具	Apache Maven（工程另附 javac 编译脚本）	3.9.9
数据库服务器	MySQL Community Server	8.0.44
JDBC 驱动	mysql-connector-java	8.0.28
数据库客户端	Navicat Premium 17	17.0.4
界面技术	Java Swing（JDK 内置）	JDK 21

工程根目录为 library-app，源程序共 21 个 Java 文件、约 2100 行代码，编译后生成的 library-app.jar 约 49 KB。

三、算法或原理分析（实验内容）

3.1 系统需求与总体设计

图书管理系统面向图书管理员，需要完成对图书信息、读者信息以及两者之间借阅关系的管理。程序的功能需求如表 1 所示。

表 1 程序功能需求

功能模块	功能描述	对应的数据库对象
图书信息管理	图书信息的录入、修改、删除与模糊查询	book 表；sp_book_insert 等 3 个存储过程
读者信息管理	读者信息的录入、修改、删除与模糊查询	reader 表；sp_reader_insert 等 3 个存储过程
借阅管理	办理借书与还书，查看借阅明细和未归还清单	borrow 表；sp_borrow_book、sp_return_book；视图 v_borrow_detail
统计查询	图书借阅排行、读者借阅统计、出版社统计、月度趋势	视图 v_book_stat、v_reader_stat 及聚合查询

程序采用三层体系结构：表示层负责与用户交互，业务逻辑层负责合法性检查与异常转换，数据访问层负责与数据库打交道。数据库层则沿用实验一的设计成果，把主键、外码、ENUM 取值和业务规则（触发器）等完整性约束交给数据库服务器实施。

3.2 系统功能结构

按照自顶向下、逐层分解的原则，图书管理系统划分为图书信息管理、读者信息管理、借阅管理和统计查询四个功能模块，每个模块再细分为若干子功能，如图 1 所示。

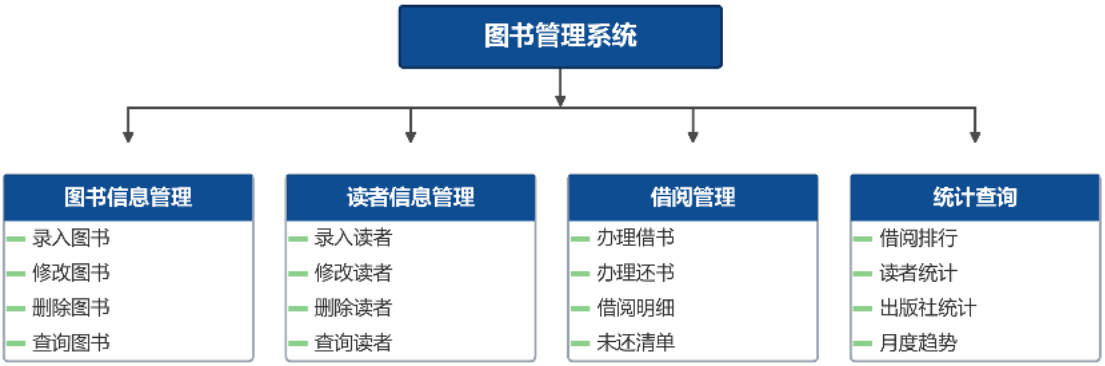


图 1　图书管理系统功能结构图

3.3　程序分层体系结构

程序分为表示层、业务逻辑层、数据访问层和数据库层四层，层与层之间只能由上向下调用，下层不能反向依赖上层。这样划分的好处是：界面代码中不出现任何 SQL 语句，数据库结构的变化只影响数据访问层，便于维护和替换。程序的分层体系结构如图 2 所示。

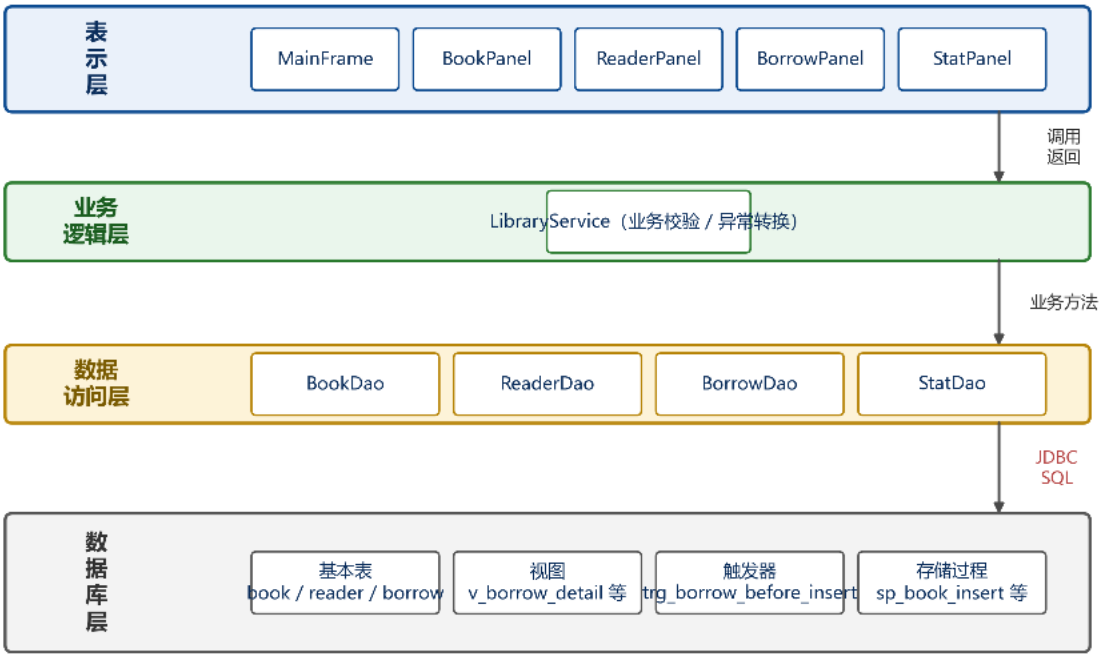


图 2　程序分层体系结构图

3.4　JDBC 访问数据库的原理

JDBC（Java Database Connectivity）是 Java 提供的数据库访问标准接口，由一组 Java 类和接

口组成。使用 JDBC 访问数据库一般分为五步：

1. 加载并注册数据库驱动：Class.forName(“com.mysql.cj.jdbc.Driver”)；

2. 建立连接：DriverManager.getConnection(url, user, password)；

3. 创建执行对象：普通查询用 PreparedStatement，调用存储过程用 CallableStatement；

4. 执行语句并处理结果集 ResultSet；

5. 关闭资源，释放连接。

本程序在实现时特别注意了以下三点：

1. 查询一律使用 PreparedStatement 并对参数使用占位符 ?，由驱动程序预编译，既提高执行效率，又避免 SQL 注入；

2. 录入、修改、删除等写操作全部通过 CallableStatement 调用实验一建立的存储过程，语法为 {call 过程名(?,?)}，把完整性检查留在数据库一侧；

3. 所有连接、语句和结果集都用 try-with-resources 语法管理，方法结束时会自动关闭，不会因为异常而泄漏数据库连接。

3.5 关键类设计

程序按包组织，各包与主要类的职责如表 2 所示。

表 2 主要类及其职责

包	类	主要职责
com.library.library	App	程序入口，设置界面外观并启动主窗口
util	DBUtil	读取 db.properties，加载驱动，提供连接与资源关闭
entity	Book / Reader / Borrow BookStat / ReaderStat	与基本表和视图对应的实体类（POJO）
dao	BookDao	图书增删改查，调用 sp_book_insert / sp_book_update / sp_book_delete
dao	ReaderDao	读者增删改查，调用 sp_reader_insert 等三个存储过程
dao	BorrowDao	借书还书，调用 sp_borrow_book / sp_return_book；查视图 v_borrow_detail
dao	StatDao	统计查询，读取视图 v_book_stat、v_reader_stat 与聚合结果
service	LibraryService	业务合法性检查、把 SQLException 转换为友好提示
ui	MainFrame	主窗口：标题栏、四个功能选项卡与状

		态栏
ui	BookPanel 等四个面板	四个功能选项卡的界面与事件处理
ui	BarChartPanel	用 Java2D 绘制统计结果条形图
ui	UIUtil	对话框、表格模型与统一界面样式等公共工具

3.6 程序流程图

图书信息录入的处理过程是：输入图书信息后先在业务层做必填项和价格合法性检查，通过后才调用业务方法；业务方法内部通过数据访问层调用存储过程 sp_book_insert；如果数据库因为主键重复等原因拒绝写入，程序捕获异常并弹出提示，用户修改后可以重新提交。流程图如图 3 所示。

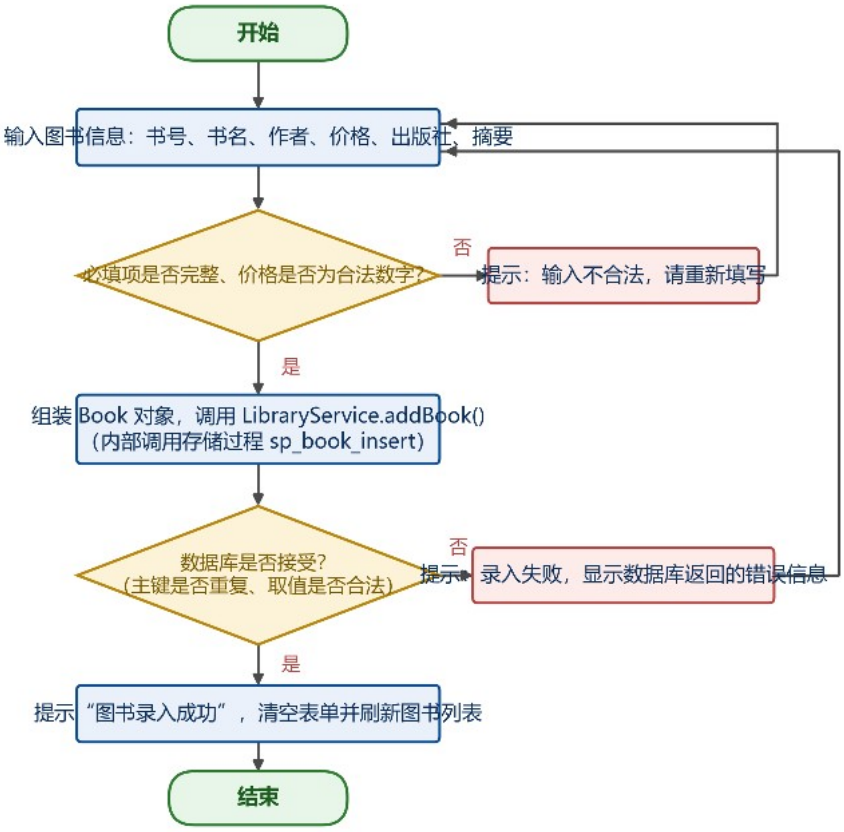


图 3 图书信息录入流程图

借书的处理过程如图 4 所示。借书请求最终交由存储过程 sp_borrow_book 处理，在写入借阅表之前会依次经过触发器校验和外键校验。校验不通过时，数据库用 SIGNAL SQLSTATE '45000' 抛出错误，Java 端捕获 SQLException 后把数据库返回的中文消息显示给用户。

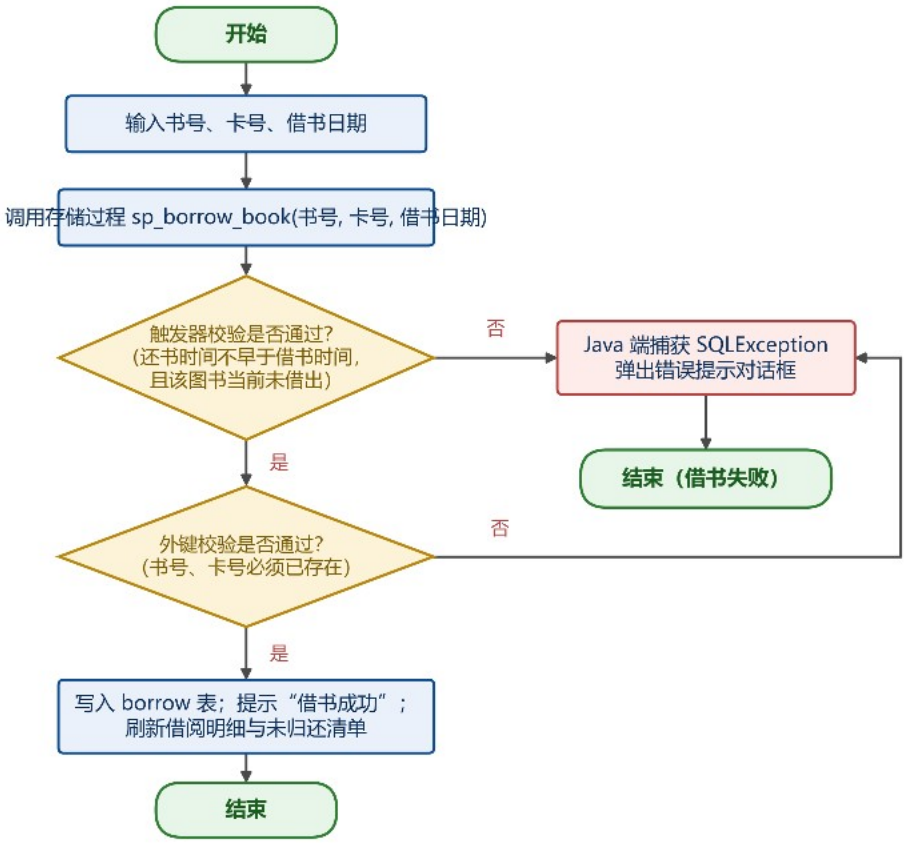


图 4　借书业务流程图

四、程序代码或实现过程

4.1　工程文件结构

工程按 Maven 的标准目录组织，源代码放在 src/main/java 下，配置文件放在 src/main/resources 下，第三方 JDBC 驱动放在 lib 下。

```
library-app/
├─ pom.xml                      Maven 工程描述文件
├── 编译.bat / 运行程序.bat          一键编译与运行脚本
├─ lib/
│   └─ mysql-connector-java-8.0.28.jar     MySQL JDBC 驱动
├─ src/main/resources/
│   └─ db.properties             数据库连接参数
├─ src/main/java/com/library/library/
│   ├─ App.java                  程序入口
│   ├─ util/DBUtil.java          JDBC 连接工具类
│   ├─ entity/                   Book、Reader、Borrow、BookStat、ReaderStat
│   ├─ dao/                      BookDao、ReaderDao、BorrowDao、StatDao
```

```
│   ├─ service/LibraryService.java
│   └─ ui/                    MainFrame、BookPanel、ReaderPanel、
│                             BorrowPanel、StatPanel、BarChartPanel、UIUtil
└─ tools/                     SmokeTest、ScreenShotDriver（测试与截图工具）
```

4.2 数据库连接配置与工具类

数据库连接参数全部写在配置文件 db.properties 中，源程序不出现任何硬编码的口令。字符集指定为 utf8，并关闭 SSL、指定时区，避免出现中文乱码和时区警告。

```
# ============================================================
#  图书管理系统 数据库连接配置
#  数据库由《实验一 建立相关数据库》创建
# ============================================================
jdbc.driver=com.mysql.cj.jdbc.Driver
jdbc.url=jdbc:mysql://127.0.0.1:3306/library_db?
useUnicode=true&characterEncoding=utf8&useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=Asi
a/Shanghai
jdbc.username=root
jdbc.password=123456
```

DBUtil 类在静态代码块中读取配置、加载驱动，并对外提供 getConnection() 方法。由于驱动只需加载一次，放在静态块中既保证只执行一次，又保证在使用前完成。

```
public static Connection getConnection() throws SQLException {
        return DriverManager.getConnection(
                CFG.getProperty("jdbc.url"),
                CFG.getProperty("jdbc.username"),
                CFG.getProperty("jdbc.password"));
    }
```

4.3 数据访问层

数据访问层是程序与数据库之间的唯一通道。查询使用 PreparedStatement，写操作通过 CallableStatement 调用实验一建立的存储过程。下面以 BookDao 为例。

（1）按关键字模糊查询图书

```
public List<Book> search(String keyword) throws SQLException {
        boolean has = keyword != null && !keyword.isBlank();
        String sql = "SELECT " + COLS + " FROM book"
                   + (has ? " WHERE book_no LIKE ? OR title LIKE ? OR author LIKE ? OR publisher
LIKE ?" : "")
                + " ORDER BY book_no";
        List<Book> list = new ArrayList<>();
```

```
        try (Connection conn = DBUtil.getConnection();
             PreparedStatement ps = conn.prepareStatement(sql)) {
            if (has) {
                String like = "%" + keyword.trim() + "%";
                for (int i = 1; i <= 4; i++) {
                    ps.setString(i, like);
                }
            }
            try (ResultSet rs = ps.executeQuery()) {
                while (rs.next()) {
                    list.add(map(rs));
                }
            }
        }
        return list;
    }
```

（2）录入图书——调用存储过程 sp_book_insert

```
public void insert(Book b) throws SQLException {
        try (Connection conn = DBUtil.getConnection();
             CallableStatement cs = conn.prepareCall("{call sp_book_insert(?,?,?,?,?,?)}")) {
            cs.setString(1, b.getBookNo());
            cs.setString(2, b.getTitle());
            cs.setString(3, b.getAuthor());
            cs.setBigDecimal(4, b.getPrice());
            cs.setString(5, b.getPublisher());
            cs.setString(6, b.getSummary());
            cs.execute();
        }
    }
```

可以看到，Java 端只负责把参数传给存储过程，主键是否重复、取值是否合法等检查全部由数据库完成。如果书号已经存在，sp_book_insert 会触发主键冲突，JDBC 抛出 SQLException，消息内容为 Duplicate entry ...，程序捕获后原样显示给用户。

（3）借书——调用存储过程 sp_borrow_book

```
public void borrow(String bookNo, String cardNo, LocalDate date) throws SQLException {
        try (Connection conn = DBUtil.getConnection();
             CallableStatement cs = conn.prepareCall("{call sp_borrow_book(?,?,?)}")) {
            cs.setString(1, bookNo);
            cs.setString(2, cardNo);
            cs.setDate(3, Date.valueOf(date));
            cs.execute();
```

```
        }
    }
```

4.4 业务逻辑层

业务逻辑层位于界面与数据访问之间，一方面做输入合法性检查，另一方面把数据库异常转换成用户看得懂的中文提示，从而使界面代码保持简洁。

```
private void checkBook(Book b) throws BusinessException {
        if (isBlank(b.getBookNo())) {
            throw new BusinessException("书号不能为空。");
        }
        if (isBlank(b.getTitle())) {
            throw new BusinessException("书名不能为空。");
        }
        if (isBlank(b.getAuthor())) {
            throw new BusinessException("作者不能为空。");
        }
        if (b.getPrice() == null || b.getPrice().compareTo(BigDecimal.ZERO) < 0) {
            throw new BusinessException("价格必须是不小于 0 的数字。");
        }
    }

public void addBook(Book b) throws SQLException, BusinessException {
        checkBook(b);
        bookDao.insert(b);
    }
```

4.5 图形界面

界面用 Java Swing 实现。主窗口 JFrame 中放一个 JTabbedPane，把四个功能面板组织成四个选项卡，底部是状态栏，显示数据库连接信息和各表的数据量。以图书管理面板的录入按钮为例，其事件处理如下。

```
private void doInsert() {
        try {
            service.addBook(readForm());
            UIUtil.info(this, "图书录入成功。");
            clearForm();
            reload();
        } catch (Exception ex) {
            UIUtil.error(this, "录入失败：\n" + ex.getMessage());
        }
```

```
    }
```

这里把所有异常统一交给 catch 块处理，并调用 UIUtil.error 弹出错误对话框。因为数据库返回的消息本身就是中文（如“该图书尚未归还，不能重复借出”），所以用户看到的提示是准确而具体的。

4.6 编译、打包与运行

工程同时提供两种构建方式：一是用 Maven 执行 mvn package；二是在没有网络的环境下，直接用 JDK 自带的 javac 编译并打包，脚本已经写好，双击 编译.bat 即可。运行程序时双击 运行程序.bat，或者执行下面的命令。

```
:: 编译
javac -encoding UTF-8 -d target\classes -cp lib\mysql-connector-java-8.0.28.jar @target\sources.txt
jar --create --file target\library-app.jar --main-class com.library.library.App -C target\classes .

:: 运行
java -cp "target\library-app.jar;lib\mysql-connector-java-8.0.28.jar" com.library.library.App
```

运行前需要先启动 MySQL 服务器（双击实验一提供的 启动MySQL服务.bat）。如果数据库没有启动，状态栏会显示“数据库连接失败”及具体原因，而不会导致程序崩溃。

五、结果分析

5.1 程序运行界面

启动 MySQL 服务器后运行程序，主窗口正常打开，底部状态栏显示数据库连接成功，并统计出当前库中的数据量：图书 24 种、读者 18 人、借阅记录 128 条（其中未归还 24 条），与实验一建立的数据完全一致。程序主界面如图 5 所示。

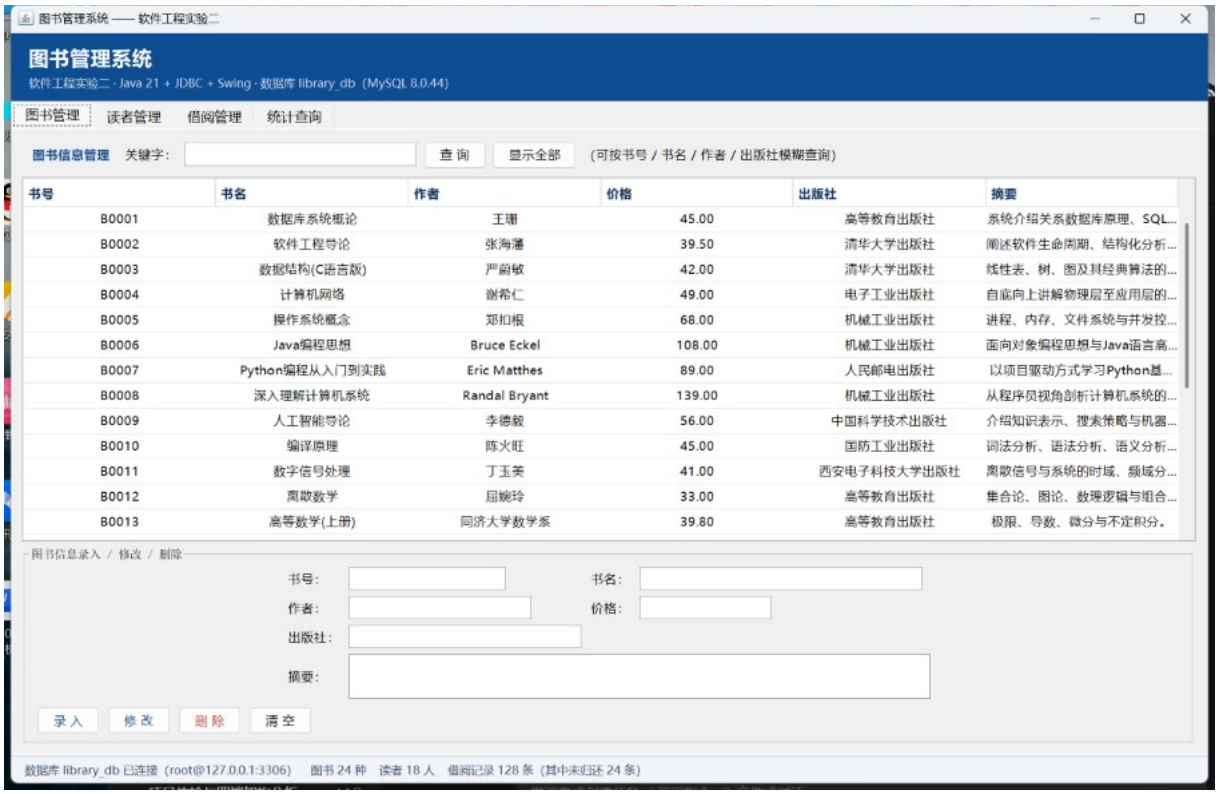


图 5　系统主界面——图书管理

5.2　图书信息管理功能

图书管理选项卡上方是查询工具条，中间是图书列表，下方是录入 / 修改 / 删除表单。单击表格中的某一行，该行数据会自动填入下方表单，修改后点“修 改”即可保存，不需要手工重新输入书号。

测试过程与结果如下：

1. 查询：在关键字框输入“数据”，单击“查 询”，列表正确筛出 《数据库系统概论》和《数据结构(C 语言版)》2 条记录，说明按书名模糊查询生效；

2. 录入：填入书号 B0025、书名《软件测试技术》、作者朱少民、价格 49.00、出版社人民邮电出版社，单击“录 入”，提示“图书录入成功”，列表刷新后出现该书；

3. 修改：选中 B0025 后把书名改为《软件测试技术（第 2 版）》、价格改为 55.00，单击“修改”，提示修改成功，表格中的书名与价格同步更新；

4. 删除：选中 B0025，单击“删 除”，确认后提示删除成功，该书从列表中消失；

5. 异常处理：若书号重复，数据库返回主键冲突，程序弹出“录入失败”对话框并显示 Duplicate entry ’B0001’ for key ’book.PRIMARY’，程序不会崩溃。

5.3　读者信息管理功能

读者管理选项卡的布局与图书管理一致，其中性别、类型、级别三项使用下拉框选择，从界面上就限制了取值范围，与数据库中 ENUM 和业务约定保持一致。界面如图 6 所示。

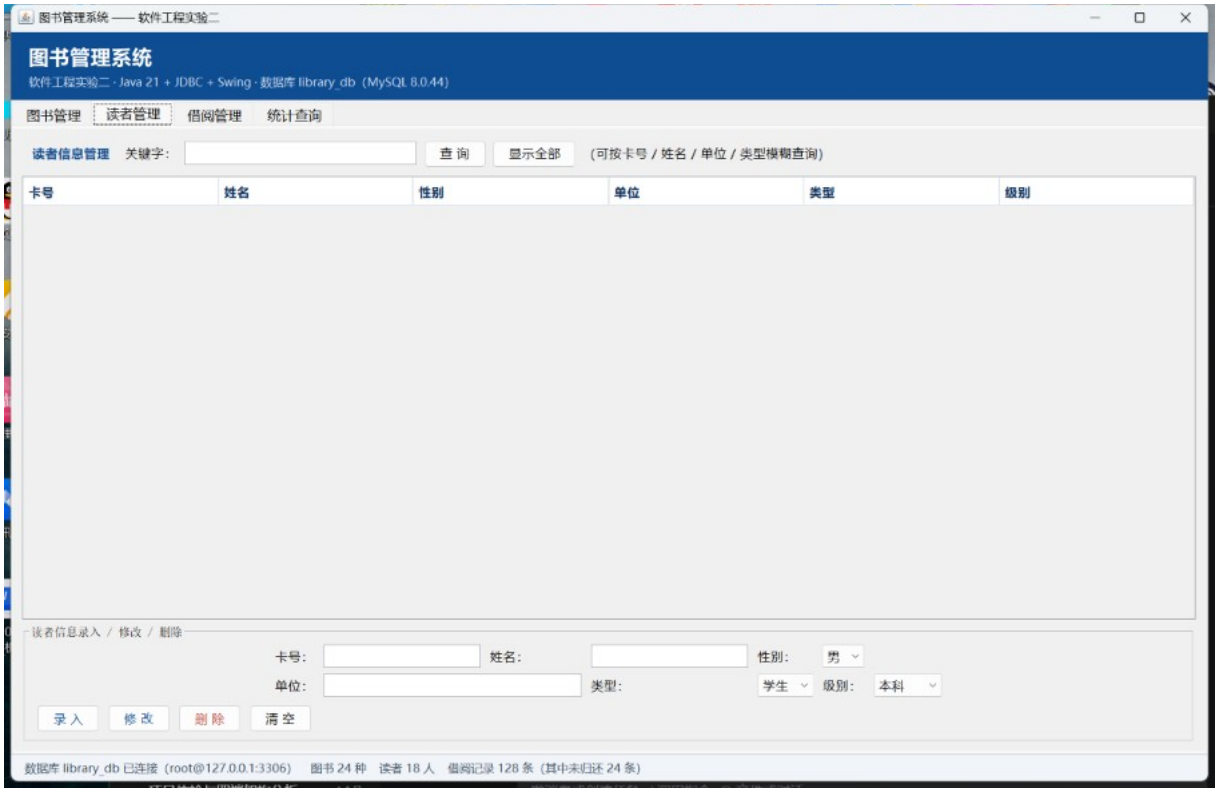


图 6　读者信息管理界面

读者信息的录入、修改、删除分别调用存储过程 sp_reader_insert、sp_reader_update 和 sp_reader_delete。测试结果与图书管理相同，三项操作均正确执行；当读者尚有图书未归还时执行删除，数据库会返回“该读者尚有图书未归还，不能删除”，程序把这条消息原样显示给用户，避免了误删。

5.4　借阅管理功能

借阅管理选项卡分为三部分：上方是查询工具条和“只显示未归还”复选框，中间是借阅明细列表，下方左右并列“办理借书”和“办理还书”两个区域。借阅日期默认为当天，也可以手工指定。界面如图 7 所示。

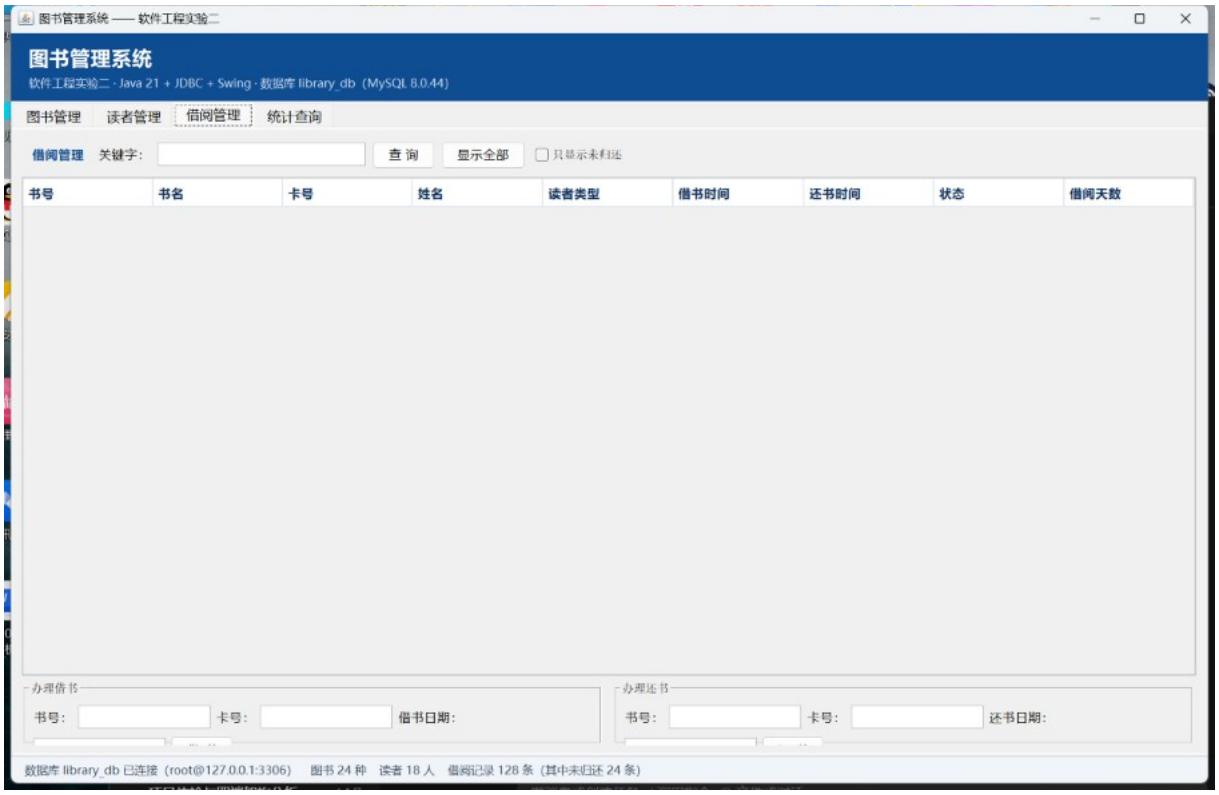


图 7　借阅管理界面

借阅明细列表的数据来自实验一建立的视图 v_borrow_detail，一条记录同时显示书号、书名、卡号、姓名、读者类型、借书时间、还书时间、状态和借阅天数。勾选“只显示未归还”后，列表只保留还书时间为空的记录，如图 8 所示，方便管理员快速掌握在借情况。

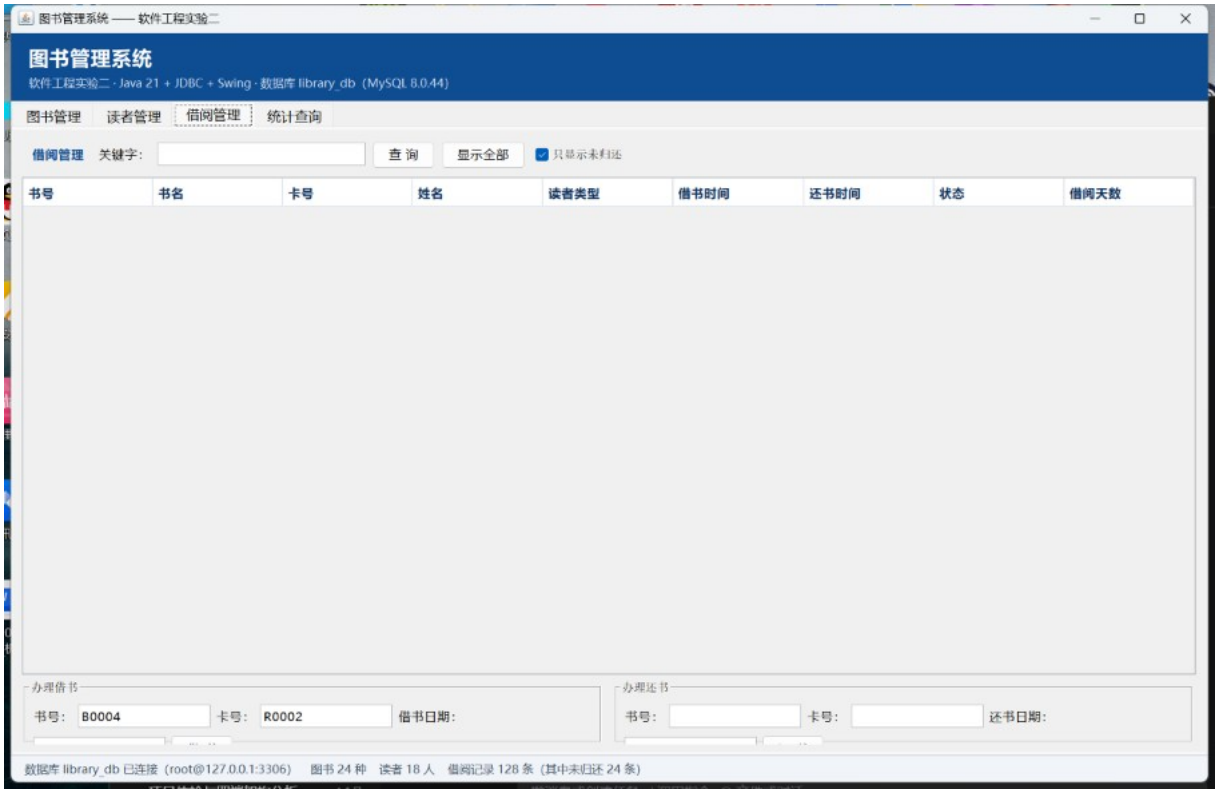


图 8　借阅管理——未归还清单

5.5　统计查询功能

统计查询选项卡提供五个统计项目，单击按钮后上方表格显示明细数据，下方用 Java2D 绘制的条形图直观呈现数量对比。

图 9 是图书借阅次数排行，数据来自视图 v_book_stat。排在前三位的是《数据结构(C 语言版)》8 次、《离散数学》7 次和《大学英语精读(第三册)》7 次，都是使用量较大的课程教材。

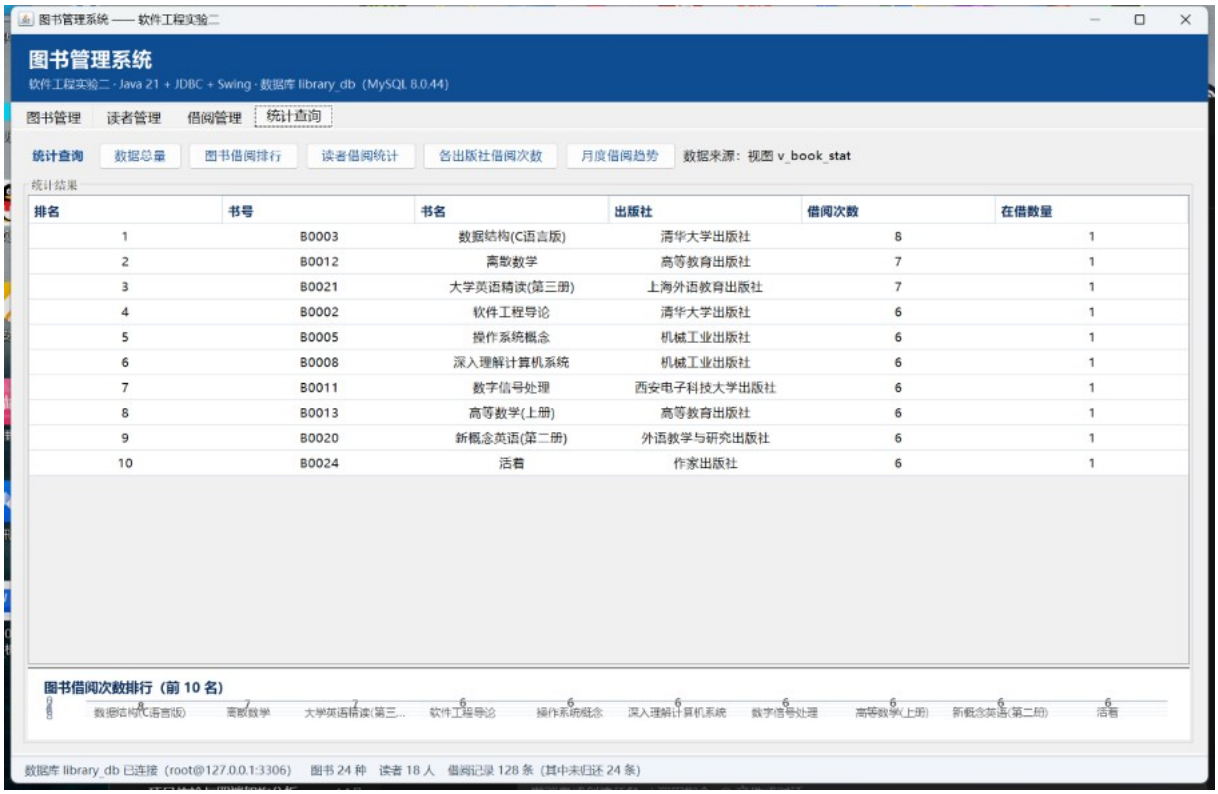


图 9　统计查询——图书借阅排行

图 10 是月度借阅量变化趋势。3 月开学季借阅量最高（30 次），7 月暑假回落到 13 次，8 月下旬随新学期临近又回升到 19 次；9 月只统计到 9 月 20 日，8 次的结果并不代表全月水平。

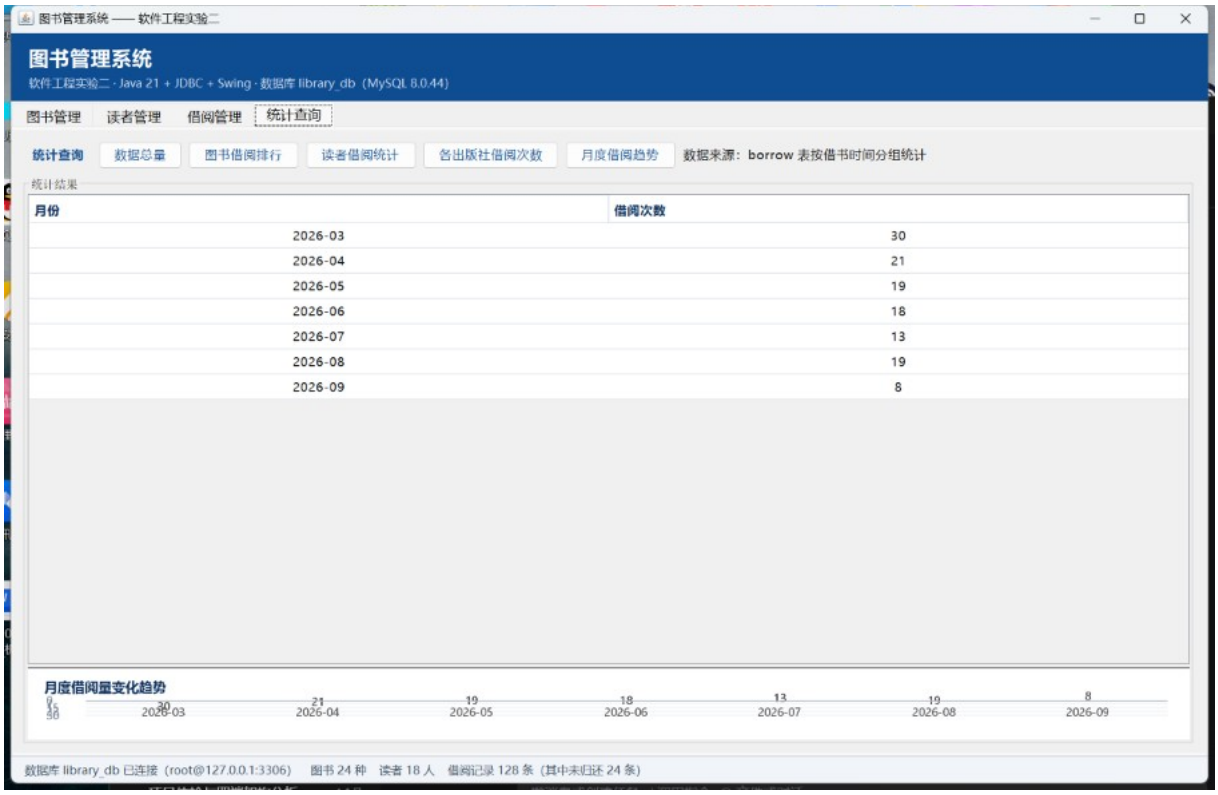


图 10　统计查询——月度借阅量变化趋势

图 11 是各出版社图书的借阅次数与占比。高等教育出版社的图书被借阅 32 次，占 25.0%，居第一位，其次是机械工业出版社 17 次、清华大学出版社 14 次，与馆藏中教材类图书占比较高相符。

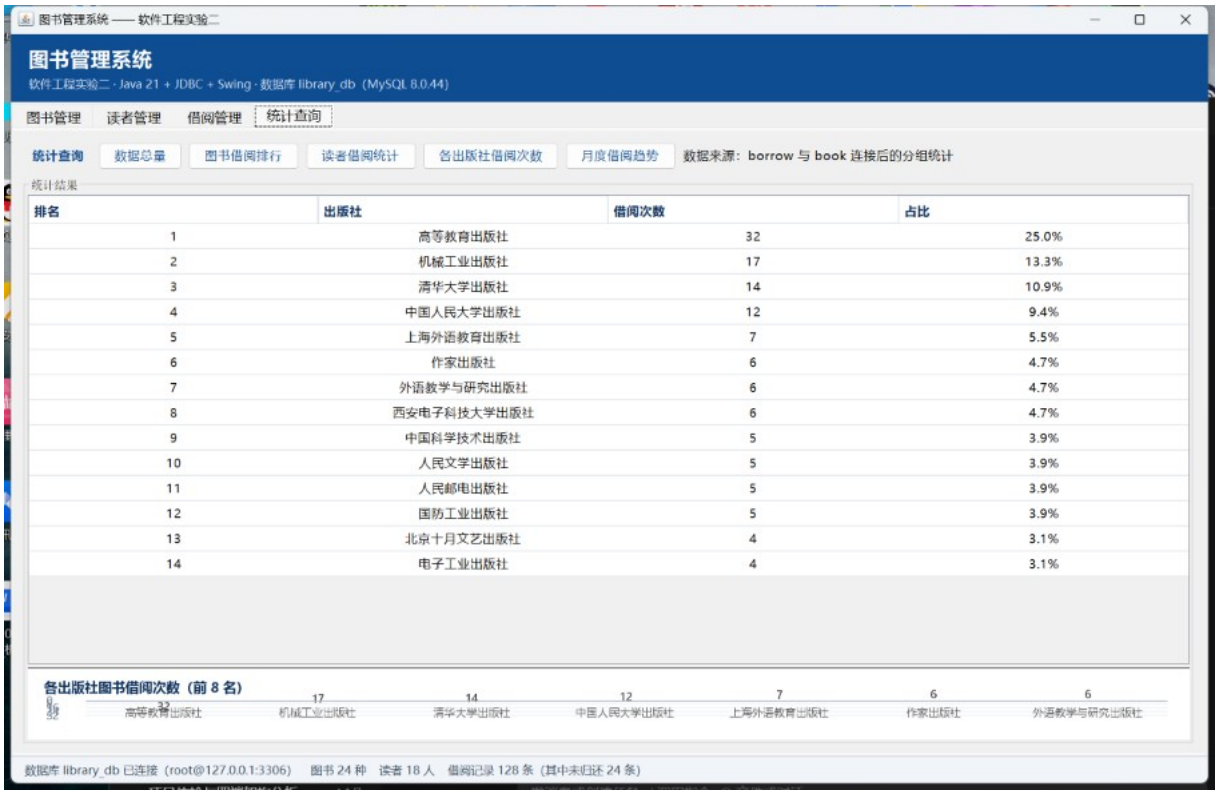


图 11　统计查询——各出版社借阅次数

5.6　数据库完整性约束在程序中的体现

本次实验最值得说明的一点，是实验一在数据库层建立的完整性约束在应用程序中得到了直接体现：程序不需要自己写检查代码，只要把数据库返回的错误消息显示出来，用户就能知道问题出在哪里。

以“把尚未归还的图书再次借出”为例：在借阅管理选项卡的办理借书区域填入一本当前未归还的图书书号（B0001）和读者卡号，单击“借　书”，触发器 trg_borrow_before_insert 在插入之前就把这次操作拦了下来，程序捕获异常并弹出对话框，提示“该图书尚未归还，不能重复借出”，如图 12 所示。

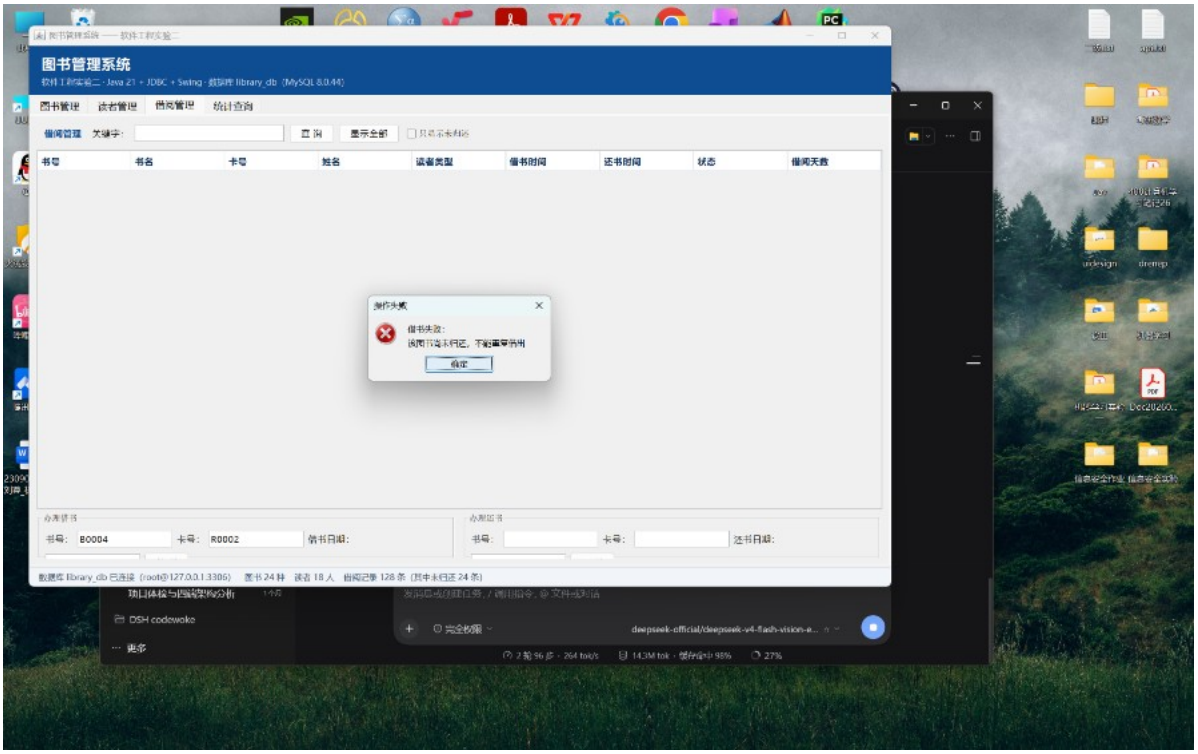


图 12　数据库触发器拒绝重复借出，程序给出友好提示

表 3　各类完整性约束在程序中的测试结果

序号	测试内容	数据库对象	程序显示的结果
1	录入已存在的书号 B0001	主键约束	录入失败：Duplicate entry 'B0001' for key 'book.PRIMARY'
2	借阅不存在的图书 B9999	外键约束 fk_borrow_book	借书失败：Cannot add or update a child row: a foreign key constraint fails ...
3	读者性别填写非法值	ENUM 取值约束	录入失败：Data truncated for column 'gender' at row

			1
4	把未归还的图书再次借出	触发器 trg_borrow_before_insert	借书失败：该图书尚未归还，不能重复借出
5	删除尚有图书未归还的读者	存储过程 sp_reader_delete	删除失败：该读者尚有图书未归还，不能删除

5.7 命令行功能测试

除了界面测试，另外编写了一个命令行测试程序 SmokeTest，直接调用业务层接口，把主要功能的执行结果打印出来，便于快速回归验证。运行结果如下。

```
总览[图书,读者,借阅,未还] = [24, 18, 128, 24]
图书全部      = 24
图书查询'数据' = 2
读者全部      = 18
借阅明细      = 128
未归还        = 24
借阅排行前3  = 3
    数据结构(C语言版) / 清华大学出版社 / 借阅 8 次 / 在借 1
    离散数学 / 高等教育出版社 / 借阅 7 次 / 在借 1
    大学英语精读(第三册) / 上海外语教育出版社 / 借阅 7 次 / 在借 1
读者统计:
    学生 本科 人数=7 借阅=53 人均=7.57
    学生 硕士 人数=4 借阅=40 人均=10.0
    教师 教职工 人数=4 借阅=25 人均=6.25
    学生 博士 人数=2 借阅=7 人均=3.5
    职工 教职工 人数=1 借阅=3 人均=3.0
出版社种类    = 14 首位=高等教育出版社=32
月度统计      = {2026-03=30, 2026-04=21, 2026-05=19, 2026-06=18, 2026-07=13, 2026-08=19, 2026-09=8}
录入后查询    = 1
修改后        = Java核心技术 卷I / 159.00
触发器拦截    = 该图书尚未归还，不能重复借出
主键拦截      = Duplicate entry 'B0001' for key 'book.PRIMARY'
删除后查询    = 0
总览[图书,读者,借阅,未还] = [24, 18, 128, 24]
SMOKE_OK
```

测试结果显示：数据库连接正常，三张表的数据量与实验一一致（24 / 18 / 128）；存储过程调用成功，录入－修改－删除三步操作后数据恢复到测试前的状态；触发器和主键约束都正确拦截了非法操作，并返回了预期的中文提示。最后打印 SMOKE_OK，表示全部测试通过。

5.8 实验结论与心得体会

1. 本实验完成了从数据库设计到应用程序实现的完整过程。实验一给出的 E-R 图、数据字典和数据流图为程序提供了清晰的依据：数据流图中的每一个加工，基本对应程序中的一个功能模块；每一个数据存储，对应数据库中的一张表。这说明规范的设计文档确实能够直接指导编码。

2. 分层设计带来的好处在实际编码中体会很深。界面代码里没有一条 SQL 语句，全部通过 LibraryService 和 DAO 完成；当需要改变查询条件时，只修改 DAO 中的一条语句即可，界面代码完全不用动。

3. 把完整性约束放在数据库层是一种有效做法。触发器、外键、ENUM 和存储过程在测试中都成功拦截了非法数据，程序只需要把数据库返回的消息显示出来即可。这样即使以后换成网页前端或手机客户端，规则依然有效，不会因为换了一个入口就出现脏数据。

4. 通过本次实验也熟悉了 JDBC 编程的规范做法：用 PreparedStatement 防止 SQL 注入、用 CallableStatement 调用存储过程、用 try-with-resources 管理资源、把连接参数放在配置文件里。这些都是实际开发中必须遵守的基本要求。

5. 程序还存在一些可以改进的地方：目前每次操作都单独建立一次数据库连接，数据量大时效率不高，可以引入连接池（如 HikariCP）加以优化；界面在批量查询时会有短暂卡顿，可以把耗时操作放到 SwingWorker 后台线程中执行；此外还可以增加用户登录与权限管理、图书超期提醒、借阅记录导出 Excel 等功能。

教师签字		年　　月 日	